Generative AI Systems
Design Lecture
07/04/2026

05
Image Captioning

Introduction

Image captioning is the process of generating textual descriptions for an image. The generated text, also known as caption, should accurately utilize the objects in the image and their attributes.

Image captioning has multiple applications. For example, on social media platforms, it automatically suggests image captions, helping users share photos with ease. In a similar vein, it generates captions for product images, thus enriching the shopping experience.

Figure 1: Image Captioning Example

Clarifying Requirements

Here is a typical discussion between a product manager and an ML engineer.

Candidate: There are various types of images, including general everyday images and domain-specific images such as medical images. Which specific type do you want to focus on for this image captioning project?
Interviewer: Stick.
Candidate: Are there any specific applications or use cases we are targeting with this system?
Interviewer: We are ongoing two independent projects. One project focuses on the visually impaired.
Candidate: Since this image captioner will be used for visual needs augmentation, the captions should not be long and detailed. Is this a correct assumption?
Interviewer: Great idea. The captions should be short, but descriptive and clear.
Candidate: Should the system support multiple languages, or will it focus only on English?
Interviewer: Let's focus on English only.
Candidate: What is the environment and size diversity of the dataset?
Interviewer: The dataset consists of 1 million high-quality image-caption pairs focused on everyday images.
Candidate: Does the dataset contain any inherent biases we need to be aware of?
Interviewer: The dataset is pre-processed. There might be unknown biases in the pre-trained backbones, and some captions might be noisy or inaccurate. Additionally, capacity for more diversity may be needed.
Candidate: Is real-time captioning required?
Interviewer: The system should generate a caption quickly, though real-time speed is not necessary. A latency of 1-2 seconds is acceptable.
Candidate: Does ensuring the system handles diverse lighting conditions and varied backgrounds is a crucial factor?
Interviewer: Yes, ensuring the system performs well across various lighting conditions and backgrounds is important.
Candidate: I assume the system should avoid generating biased captions or captions with offensive assets. Is that a fair assumption?
Interviewer: Yes, that is crucial to ensure our system remains fair and inclusive.
Candidate: What are the typical image dimensions? Very small images can be pixelate, leading to incorrect captions.
Interviewer: Let's assume the system supports images of any size with a minimum resolution of 256 x 256 pixels.

Frame the Problem as an ML Task

Specifying the system's input and output

The input to an image captioning system is an image. The image is processed by the model to generate a descriptive caption. The output, therefore, is a text that accurately describes the content of the image.

Figure 2: A black dog is running in a grassy field.

Choosing a suitable ML approach

The image captioning problem introduces a unique challenge: an ML model requires visual understanding to process the input image, language understanding to generate coherent text, and the ability to bridge these two modalities to accurately describe the visual elements in the image.

Deep learning is the ideal approach for building such multi-modal systems to scale and encounter novelties. Similar to language translation, it serves as a robust all-in-one end-to-end solution. Architectures can treat the image as a new "language". In the context, specifically, this simply two main components, each handling one modality:

1. Image encoder
2. Text decoder

Image encoder

The image encoder is responsible for understanding the visual content of the image and encoding the image into a lower-dimensional representation.

Text decoder

The text decoder uses the encoded visual information from the image encoder to generate a descriptive caption.

Figure 3: Image captioning components

Choosing a suitable ML approach

The image captioning problem introduces a unique challenge: an ML model requires visual understanding to process the input image, language understanding to generate coherent text, and the ability to bridge these two modalities to accurately describe the visual elements in the image.

Deep learning is the ideal approach for building such multi-modal systems to scale and encounter novelties. Similar to language translation, it serves as a robust all-in-one end-to-end solution. Architectures can treat the image as a new "language". In the context, specifically, this simply two main components, each handling one modality:

1. Image encoder
2. Text decoder

Image encoder

The image encoder is responsible for understanding the visual content of the image and encoding the image into a lower-dimensional representation.

Text decoder

The text decoder uses the encoded visual information from the image encoder to generate a descriptive caption.

Figure 4: Image captioning components

We will explore the architecture of these components in detail in the model development section. It's important to note that these components are essential for building the image captioning problem. While we focus on the encoder-decoder framework here, alternative approaches exist. For instance, a vision-only model could classify the image into predefined categories, then simply retrieve a caption. If you're interested in exploring other ML solutions, you can refer to [1], [2], [3] for a broader understanding of the image captioning landscape.

Data Preparation

In this section, we prepare the dataset to train our image captioning system.

Image	Caption
	A black dog is running in a grassy field. A brown dog in the grass. A brown dog in a field.

Figure 5: Example of image-caption dataset

The dataset comprises 400 million image-caption pairs and captions, moreover, not all images or captions are suitable for training. Let's examine data preprocessing steps.

Caption preparation

Captions can vary in length, with some opting for a concise style and others for a more descriptive approach.

- **Remove punctuation and special characters:** While removing punctuation and special characters can streamline the model's focus on the core text, it's important to consider the context. In some cases, these elements might carry meaning or convey specific information.
- **Remove rare words:** Eliminating rare words from the vocabulary helps prevent the model from learning spurious correlations. These rare words might not provide enough detail for accurate caption generation.
- **Truncate long captions:** To maintain computational efficiency and avoid memory issues, it's crucial to limit the maximum caption length. This ensures the model processes a manageable amount of text for each image.
- **Remove irrelevant captions:** To maintain high-quality training data, we filter images that exhibit conditions such as excessive blurriness, low resolution, or predominantly dark or bright pixels. Additionally, we leverage pre-trained models, such as the CLIP(Contrastive Language-Image Pre-training) model, to identify and remove images with low alignment between captions and images, ensuring relevance.
- **Remove non-descriptive captions:** Captions that primarily have a range of stop-word usages value. The inclusion of images in a different size. This is crucial because AI models typically require square inputs during training. While adjusting image dimensions can be a uniform step, it is important to preserve their original aspect ratios. To do so, we utilize three ways:
- **Resizing:** Pre-trained models like ViT and the smaller-dimension variants have strict requirements. For instance, if our target size is 224x224 and our original image is 512x256, we resize it to 224x224.
- **Padding:** Instead of resizing, we can add black pixels to reach the target dimensions. From our previous example, we center-crop the 256x256 image to 224x224.

Figure 6: Resizing followed by center-cropping

This has clear artifacts known as an image-between-text aspect ratio and fit the required size for our ML model.

Model Development

Architecture

We formulate image captioning as a multi-modal language generation task where the image encoder processes an input image and the text decoder generates a descriptive caption. In this section, we explore the architecture of the image encoder and text decoder.

Image encoder

The image encoder is responsible for processing an image and encoding the information within it.

The output of the encoder plays a pivotal role in determining the quality and specificity of the generated captions. The encoder's role is to extract a rich set of visual features, representing the entire image or a specific region of interest, in a sequence of vectors, where each vector corresponds to a specific region or aspect of the image. The choice between these two approaches has significant implications for the model's performance. The first approach, while simpler and computationally less intensive, might struggle with capturing fine-grained details and intricate relationships.

Figure 8: Image encoder outputting single vector vs. sequence of vectors

When the encoder produces a single vector for each image, it effectively condenses the entire image into one vector. This vector then serves as the initial input for the text decoder, guiding the generation of the caption. While this approach offers simplicity and speed, it can be challenging for architectures requiring more sophisticated cross-modal interaction mechanisms. In such cases, the latter approach is more suitable, as it provides a sequence of vectors, each of which can be particularly beneficial for upcoming mechanisms like attention, allowing the model to dynamically focus on different aspects of the image during caption generation. This granular representation allows the model to capture more nuanced details and relationships within the image, potentially leading to more accurate, contextual, and informative captions. Let's explore two prominent image encoder architectures: Convolutional Neural Networks (CNNs) and transformer-based image encoders.

CNN-based

Convolutional Neural Networks (CNNs) are traditionally used for image-recognition tasks. CNNs excel at capturing spatial relationships in images through the concept of convolutional filters. These filters slide across the image, detecting specific patterns and features at different scales.

CNN-based encoders process the input image and output a grid of feature vectors. For example, as shown in Figure 9, our input image passes through the CNN, producing a feature tensor of size 8 x 8 x 1024. Here, 1 represents the channel size, which depends on the specific CNN architecture. The 8 x 8 dimension signifies the spatial dimensions of the feature map, indicating the presence of features in 64 (8 x 8) distinct regions. To utilize this as a sequence, we flatten the spatial dimensions while preserving the features from each of the 64 positions in the 8 x 8 grid into a sequential format.

Figure 9: CNN-based image encoding

Transformer-based image encoders, popularized by natural language processing, have recently been adapted for image encoding with significant success. In the architecture, a Transformer analyzes images, extracts features, and encodes them into a sequence of embeddings. Specifically, a Transformer-based image encoder consists of:

- Patching
- Positional encoding
- Transformer

Figure 10: Transformer-based image encoding

Patching

Since Transformers work with sequences, the image should first be converted into a sequence. The process involves three steps:

1. Divide the image into fixed-size patches
2. Flatten the patches
3. Create a linear each patch

For example, a 256 x 256 image patch is divided into patches of 16 x 16. These patches are flattened into 4096-sized vectors and linearly projected into embedding vectors of size 1, where 1 is the desired embedding size.

Figure 11: Transformer-based image encoding

Positional encoding

Positional encoding assigns position information to each patch, specifying where each patch was located in the original image. This helps Transformers understand positions within the sequence.

Positional encoding can be implemented in various ways. Let's briefly explore the following variations:

- 1D vs. 2D positional encoding
- Learnable vs. fixed positional encoding

1D vs. 2D positional encoding

1D positional encoding treats patches as a single dimension (position in the sequence) or a dimensional vector, where it usually uses the Transformer's hidden dimension. This approach is simple and computationally efficient, but it does not explicitly capture spatial relationships between patches. In contrast, 2D positional encoding assigns two dimension vectors, one for the position of each patch in a horizontal dimension and another for the vertical dimension, effectively capturing the spatial relationships between patches.

2D positional encoding, on the other hand, helps Transformers understand the relative and absolute positions in the image grid, crucial for dimensional understanding. This enhanced spatial awareness can lead to improved performance in vision tasks.

Figure 12: 1D vs. 2D positional encoding

Learnable vs. fixed positional encoding

Fixed positional encoding employs predefined functions to assign unique vectors to each patch. Standard vision transformers utilize a 1D or 2D dimensional vector. In the fixed approach, positional encodings are generated using a fixed function such as sine or cosine. For more details, refer to Chapter 3.

There is other to learn solution when choosing between 1D vs. 2D and learnable vs. fixed positional encoding. While Swin Transformer (ViT) [5] uses learnable 1D positional encoding, in practice, we often find different combinations to see which works best for a specific task.

Which architecture is suitable for our image encoder?

The choice of an image encoder architecture depends on factors like the struggle with long-range dependencies between distant regions of the image, in contrast, Transformers capture global relationships and interactions between all image patches, enhancing understanding. This allows Transformers to model complex dependencies, making them ideal for tasks that require detailed, context-aware image understanding. While CNNs have been a mainstay in image recognition, Transformers, like Swin ViT [5] and Vision Transformer, have emerged as a compelling alternative.

Text decoder

The text decoder is responsible for generating the captions. As we saw in previous chapters, a decoder-only Transformer is the standard choice for text generation. The decoder takes the encoded image information from the image encoder and a sequence of tokens corresponding to the input image. Its output is the captions, generated one at a time.

Figure 13: Providing image as a sequence of embeddings

Training

The training approach for the image captioning model is similar to the strategies discussed in previous chapters. We follow a two-stage training strategy:

1. Unsupervised pretraining
2. Supervised fine-tuning

1. Unsupervised pretraining

During this stage, the text decoder - which is a decoder-only Transformer - is trained on general data. The purpose of this stage is to equip the decoder with a robust understanding of language patterns, grammar, and contextual relationships. This knowledge is crucial for the subsequent fine-tuning stage, where the decoder will learn to generate captions based on the encoded image information.

The pretraining stage is computationally expensive. It's common practice to use existing pretrained models in lieuzes this stage and thus, significantly reduces computational costs. In this example, we can leverage a pre-trained GPT-2 Transformer such as GPT-2 Small (124 M parameters).

Similarly, the image encoder can also be derived from pretrained backbones. Instead of training an image encoder from scratch, we can leverage pretrained models such as Swin Transformer (ViT) [5].

2. Supervised fine-tuning

In this stage, we train both the image encoder and the text decoder on 400 million image-caption pairs. This large dataset ensures the model learns to generate captions effectively and the text decoder learns to understand the sequence of image embeddings and generate coherent captions.

An objective and loss function

The last decoder predicts the caption one token at a time. Consider with previous chapters, we can train from prediction as cross-entropy loss, which measures cross-entropy loss [33] to guide the training process.

Figure 14: Cross calculation over the predicted probabilities

Sampling

During sampling, the caption tokens are generated one at a time.

Figure 15: Generating a caption given an input image

While practically sampling captions, we can employ various methods, each with its unique characteristics. We can leverage such for image captioning system for the following reasons:

- **Quality:** Beam search typically generates higher quality captions, which is critical for accurately describing image content.
- **Diversity:** The stochastic nature of beam search ensures that the model always produces the same caption for the same image, which can be limiting in real-world applications.
- **Coherence:** Beam search optimizes for overall caption coherence, which is important for image captions. The subtle variations in word changes or contractions such as "A person is walking house" or "A dog is wading a person."

Evaluation

Offline evaluation metrics

During offline evaluation, we assess the performance of the trained model on a validation dataset. This is achieved by comparing generated captions with ground-truth captions, using various evaluation metrics to quantify performance.

Before explaining common metrics, let's first understand why traditional metrics like accuracy, precision, and recall are not suitable for image captioning. This task is a more robust model that is capable of describing images more accurately.

- **Compositionality:** Accurate captions provide a more thorough assessment of a model's performance. Comparing the generated caption to several correct reference captions leads to a more evaluation.

Image	Reference captions
	Morning dog with green leaves Dalmatian enjoying the day Bright day with leaves Dalmatian in a green field

Figure 16: An example of validation data.

The following metrics are commonly used in the offline evaluation of image captioning models:

- BLEU
- ROUGE
- METEOR
- CIDEr

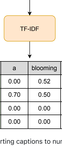
The first three metrics on the list are explored extensively in Chapter 9. In contrast, we focus on CIDEr, which has been designed specifically to evaluate image captioning models.

CIDEr

CIDEr (Consensus-based Image Description Evaluation) is a metric commonly used to evaluate the similarity of a generated caption to a set of reference captions. CIDEr aims to capture the human consensus on what an ideal image caption should be. It achieves this by considering the following factors:

1. Important captions using Term Frequency Inverse Document Frequency (TF-IDF).
2. Word order in the caption.
3. Appropriate word choice.

In the first step, we compute the generated caption and each reference caption into n-grams representations using TF-IDF. TF-IDF evaluates a word's importance in a document by considering how frequently it appears in that document and how common or rare it is across the entire corpus. These important terms are assigned higher weights in the n-gram representation. To learn more about TF-IDF, refer to [33][34].



	a	blooming	leaves	single	tulip	leaves
Ref 1: "Blooming tulip with leaves"	0.00	0.52	0.43	0.00	0.36	0.43
Ref 2: "A blooming tulip"	0.70	0.50	0.00	0.00	0.50	0.00
Ref 3: "Single tulip with leaves"	0.00	0.00	0.57	0.57	0.34	0.57
Generated: "Tulip with leaves"	0.00	0.00	0.70	0.00	0.50	0.50

Figure 15: TF-IDF converting captions to numerical representations

2. Calculate similarity

Next, we calculate the similarity between the generated caption and each reference caption. We do this by computing the cosine similarity between their TF-IDF representations.

	a	blooming	leaves	single	tulip	leaves
Ref 1: "Blooming tulip with leaves"	0.00	0.52	0.43	0.00	0.36	0.43
Ref 2: "A blooming tulip"	0.70	0.50	0.00	0.00	0.50	0.00
Ref 3: "Single tulip with leaves"	0.00	0.00	0.57	0.57	0.34	0.57
Generated: "Tulip with leaves"	0.00	0.00	0.70	0.00	0.50	0.50

Pair	Cosine similarity score
<Reference 1, Generated>	0.688
<Reference 2, Generated>	0.257
<Reference 3, Generated>	0.766

Figure 16: Calculation of cosine similarity between generated and reference captions

A higher cosine similarity score (i.e., a score closer to 1) indicates greater similarity, while a lower value (closer to 0) indicates lesser similarity.

3. Aggregate the similarity scores

Once we have the cosine similarity scores between the generated caption and each of the reference captions, we take an average of these scores. This average score reflects the overall similarity between the generated caption and the reference captions.

$$CIDEr_i = \frac{0.688 + 0.257 + 0.766}{3} = 0.570$$

The final CIDEr score is calculated by averaging the similarity scores for all generated captions in the validation dataset. This provides a single metric to evaluate the model's overall performance.

Let's see some of the pros and cons of the CIDEr metric.

Pros:

- **Consensus-based:** CIDEr emphasizes consensus by rewarding captions that are similar to multiple reference captions. This leads to a more reliable evaluation of a model's performance.
- **Sensitive to important words**: TF-IDF assigns more weight to unique words in their representation. This ensures that the CIDEr score reflects the importance of words and rewards captions that use those words.
- **Robust to different caption variations:** CIDEr is robust to different variations of generations since it is calculated based on multiple reference captions.

Cons:

- **Computationally complex:** Calculating TF-IDF representations in large datasets can be computationally expensive.
- **Sensitive to the quality of reference captions:** The quality and diversity of reference captions impact the CIDEr score. Poor references can lead to misleading evaluations.
- **Penalizes novel yet accurate captions:** CIDEr may penalize creative or novel phrases that are still accurate but are not present in the reference set.
- **Lack of semantic understanding:** CIDEr relies on TF-IDF to measure the similarity between two sentences. This might not always capture the semantic similarity when captions are textually similar but semantically different. For example, "Coffee on top of the table" and "Table on top of the coffee" might have similar TF-IDF representations due to similar words, but they are not semantically similar.

Online evaluation metrics

Online evaluation metrics are important for assessing the performance of ML systems. However, they are often not the primary focus in image captioning systems for two main reasons. First, image captioning systems are usually part of a bigger system, making it harder to collect user interaction data. Second, collecting feedback from users is challenging. Unlike tasks where we can easily measure user satisfaction, evaluating image caption quality requires subjective judgment, which, by definition, varies between users. For example, a caption might be acceptable to one user but not to another, depending on their personal interpretation of the image.

In summary, standard offline metrics remain the primary method for evaluating our image captioning system. For the few use cases where image captioning impacts user experience directly, engagement metrics and user feedback can provide valuable insights into the system's performance.

Overall ML System Design

Building an image captioning system is more than just training a model. It requires various components working together. In this section, we discuss the following key components essential for building an image captioning system:

- Image preprocessing
- Caption generator
- Post-processing

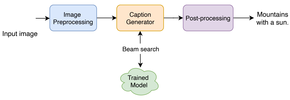


Figure 17: Image captioning overall design

Let's briefly explore each component and understand its role.

Image preprocessing

Image preprocessing is the initial step that prepares an input image for the trained model. This involves resizing images to a standard size, converting them into a consistent format, and standardizing pixel values. This step ensures that images are consistent with what the model expects as input.

Caption generator

The caption generator is the core component that produces captions based on the prepared image. This component interacts with the trained model and employs beam search to generate a coherent caption. If the cumulative probability of the generated caption falls below a predefined confidence threshold, the name suggestion is disabled; otherwise, the caption is passed to the post-processing component. This ensures that the system avoids producing irrelevant captions for ambiguous images.

Post-processing

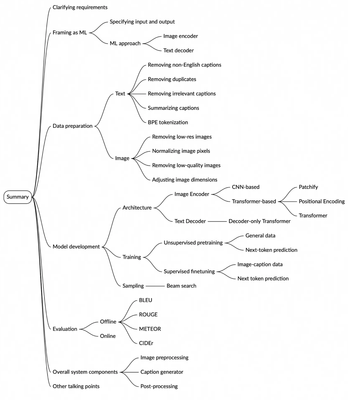
The post-processing component identifies biased terms or phrases in the caption and replaces them with neutral alternatives. This ensures fairness and inclusivity in generated captions. Additionally, it checks for the presence of offensive words and disables the name suggestion service if any are found.

Other Talking Points

If the interview finishes early, you might want to bring up the following topics:

- Extending the image captioner to support other tasks, such as visual question answering (VQA) [14].
- Adapting models to caption images from various domains [15].
- Generating captions in multiple languages using multilingual datasets and cross-lingual transfer learning [16].
- Optimization techniques for caption generation on edge devices [17].
- Generating and ranking multiple plausible captions based on relevance [18].
- Details of BLIP-2 and BLIP-3 methods and additional loss functions utilized for improving captioning [1] [2].

Summary



Reference Material

[1] BLIP-2: Bootstrapping Language-Image Pre-training with Frozen Image Encoders and Large Language Models. https://arxiv.org/abs/2301.12597.
[2] xGen-MM (BLIP-3): A Family of Open Large Multimodal Models. https://www.arxiv.org/abs/2408.08872.
[3] InternVL: Scaling up Vision Foundation Models and Aligning for Generic Visual-Linguistic Tasks. https://arxiv.org/abs/2312.14238.
[4] Meta's Llama. https://llama.meta.com/.
[5] Byte-pair encoding tokenization. https://huggingface.co/learn/nlp-course/en/chapter6/5.
[6] LAION-5B: An open large-scale dataset for training next generation image-text models. https://arxiv.org/abs/2210.08402.
[7] An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale. https://arxiv.org/abs/2010.11929.
[8] Language Models are Unsupervised Multitask Learners. https://cdn.openai.com/better-language-models/language_models_are_unsupervised_multitask_learners.pdf.
[9] Learning Transferable Visual Models From Natural Language Supervision. https://arxiv.org/abs/2103.00020.
[10] Cross-entropy. https://en.wikipedia.org/wiki/Cross-entropy.
[11] CIDEr: Consensus-based Image Description Evaluation. https://arxiv.org/abs/1411.5726.
[12] TF-IDF introduction. https://web.stanford.edu/class/cs276/19handouts/lecture6-tfidf-1per.pdf.
[13] TF-IDF. https://en.wikipedia.org/wiki/TF%E2%80%93IDF.
[14] Visual question answering introduction. https://huggingface.co/tasks/visual-question-answering.
[15] Cross-Domain Image Captioning with Discriminative Finetuning. https://arxiv.org/abs/2304.01662.
[16] Crossmodal-3600 — Multilingual Reference Captions for Geographically Diverse Images. https://research.google/blog/crossmodal-3600-multilingual-reference-captions-for-geographically-diverse-images/.
[17] Efficient Image Captioning for Edge Devices. https://arxiv.org/abs/2212.08985.
[18] Ensemble model using an image captioning and ranking example. https://cloud.google.com/dataflow/docs/notebooks/run_inference_multi_model.

Mark as Complete

Partner With Us
Be an Affiliate
Become a Contributor

Company & Legal
Our Team
Newsletter
Privacy Policy
Terms of Service
Do Not Sell or Share My Personal Information

Support
hi@bytebytego.com
Report a Bug

Resources
YouTube
Visual Dev Guides
Find Jobs
Prepare for Coding Interviews

Ask Alex

Copyright ©2022-2026 ByteByteGo Inc. All rights reserved.